2024

SQL-Driven Retail Sales and Customer Insights

Ayesha Javaid

11/17/2024

Table of Contents

Introduction: 2

Data Preprocessing: 2

Date corrections: 3

Database Structure Overview 3

ERD Diagram 4

Module 1: Sales Performance Analysis 4

Total Sales per Month Analysis 4

Average Discount Per Month 5

Module 2: Customer Behavior and Insights 6

Identifying High-Value Customers 6

Details of Customers Born in the 1990s 7

Customer Segmentation: 8

Module 3: Inventory and Product Management 9

Stock Management: 9

Inventory Movements Overview: 10

Ranking Products by Price 11

Module 4: Advanced Analytics 13

Advanced Analytics – Average Order Size 13

Advanced Analytics – Recent Restock Products 14

Seasonal Trends 15

Customer Purchase Patterns 15

SQL-Driven Retail Sales and Customer Insights

Introduction:

Understanding sales performance, customer behavior and stock behavior is quite essential for better recognition of business performance, driving growth and ensuring sustainability. This report throws light on Sales and customer Insights Analysis and examines sales, customer, and product data of a retail company to uncover valuable insights that can inform strategic decision-making.

MySQL workbench is utilized for conducting SQL queries to process and interpret the dataset which comprises customer demographics, purchase behaviors, product performance, and inventory movements. The goal of this project is to extract meaningful trends and patterns, such as identifying top-performing products, high-value customers, and inventory challenges.

Tools: SQL tools such as **CTEs**, **window functions**, **subqueries**, and **aggregation techniques** are applied in this with the aim of providing valuable recommendations that enhance business performance and optimize operational efficiency.

Data Preprocessing:

Before performing sales and customer analysis, all the primary and foreign key relationships are constructed within the tables. The datatype of all the date columns is corrected to date format from text.

```
use sales_analysis;

alter table products_data
add primary key (product_id);

alter table sales_data
add primary key (sale_id),
add foreign key (customer_id) references customers_data(customer_id),
add foreign key (product_id) references products_data (product_id);

alter table inventory_movements_data
add primary key (movement_id),
add foreign key (product_id) references products_data(product_id);

select * from inventory_movements_data;
-- disable the safe mode
set sql_safe_updates = 0;
```

Date corrections:

```
update sales_data
set sale_date = str_to_date(sale_date, "%Y-%m-%d");

alter table sales_data
modify sale_date date;
```

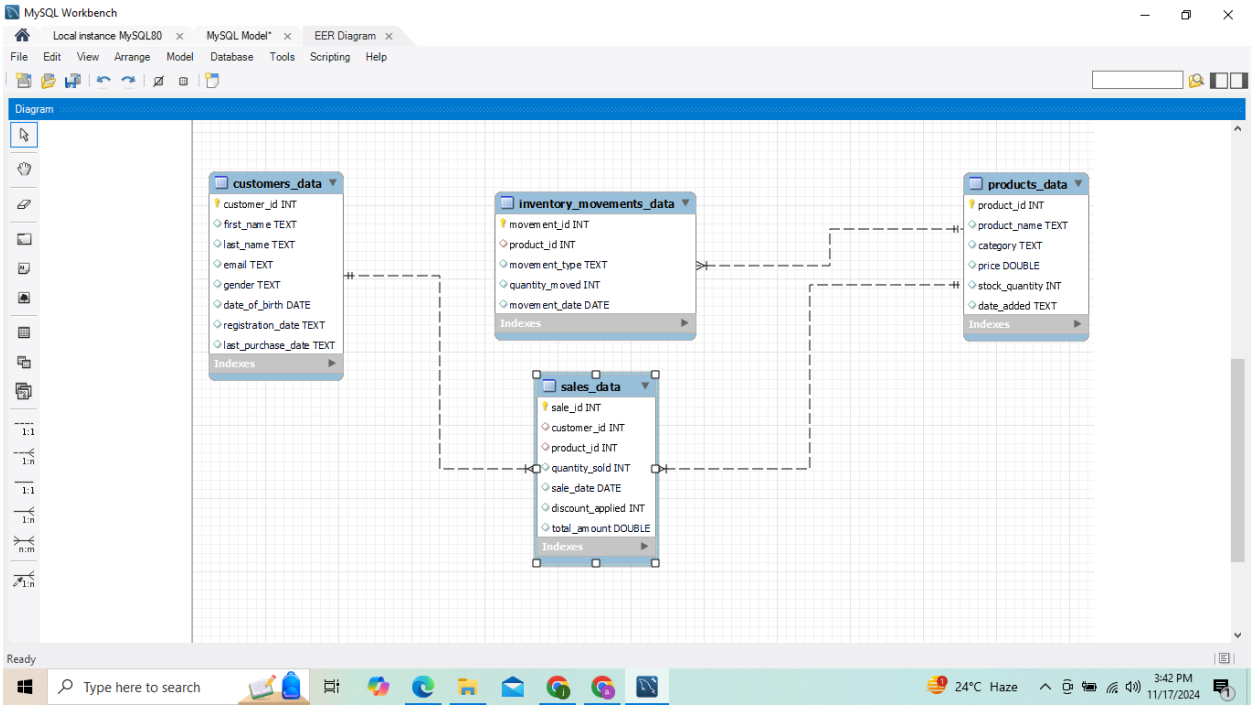
Database Structure Overview

A well-structured database is the backbone of any data-driven project. The schema consists of four interconnected tables, with relationships defined through primary and foreign keys to ensure data integrity.

Table Name	Description:
Customers table (customers_data)	This table stores all relevant customer details, including demographic information, registration dates, and their last purchase activity. This table is linked to the **Sales table (sales_data)** through the customer_id field, which enables tracking of customer purchasing behavior.
Products table (products_data)	This table holds detailed information about the company's inventory, including product categories, prices, stock levels, and the date products were added. This table connects to both the sales_data table, to track which products are sold.

Sales table (sales_data):	This table holds transactional details, such as the customer, product, quantity sold, sale date, discounts applied, and the total amount. This table acts as the central table, connecting the customers and products table to each other. It allows analysis of sales trends and customer-product interactions.
Inventory Movements table (inventory_movements_data):	This table tracks inventory adjustments over time, stock inflows (IN for restocking) and outflows (OUT for sales). Its connection to the products_data table ensures that stock levels are accurately tied to product information.

ERD Diagram



Module 1: Sales Performance Analysis

Total Sales per Month Analysis

Monthly sales trends are necessary for understanding business performance and planning future strategies. In this analysis, the total sales and revenue generated for each month are calculated using data from the Sales Table. The SQL query aggregates sales data by month. It summarizes key metrics such as the total number of units sold, and the total revenue generated.

```sql
SELECT
    DATE_FORMAT(sale_date, '%Y-%m') AS month,
    SUM(quantity_sold) AS Total_Sales,
    ROUND(SUM(total_amount), 2) AS Revenue_Generated
FROM
    Sales_data
GROUP BY
    DATE_FORMAT(sale_date, '%Y-%m')
ORDER BY
    month;
```

month	Total_Sales	Revenue_Generated
2023-11	115	22203.43
2023-12	140	30448.4
2024-01	123	22347.63
2024-02	80	15681.57
2024-03	121	24974.2
2024-04	106	24704.07
2024-05	151	32691.2
2024-06	135	27238.63
2024-07	138	32316.64
2024-08	124	30544.39
2024-09	111	21681.36
2024-10	125	30086.27
2024-11	27	6010.65

Average Discount Per Month

The average discount applied to transactions and the corresponding **total sales** generated each month is analyzed to understand the impact of discounting strategies on total sales. This query aggregates data from the Sales_data table by month.

```
SELECT
    DATE_FORMAT(sale_date, '%Y-%m') AS month,
    AVG(discount_applied) AS average_discount,
    SUM(total_amount) AS total_sales
FROM
    Sales_data
GROUP BY
    DATE_FORMAT(sale_date, '%Y-%m')
ORDER BY
    month;
```

month	average_discount	total_sales
2023-11	10.9091	22203.43
2023-12	9.8	30448.4
2024-01	9.8684	22347.63
2024-02	11.8519	15681.57
2024-03	9.5122	24974.2
2024-04	10.1316	24704.07
2024-05	9.3396	32691.2
2024-06	8.1818	27238.63
2024-07	11	32316.64
2024-08	11.4634	30544.39
2024-09	9.5238	21681.36
2024-10	10.3947	30086.27
2024-11	9	6010.65

Module 2: Customer Behavior and Insights

Identifying High-Value Customers

To determine which customers have spent the most on their purchases, I analyzed total spending for each customer by joining the customers_data and sales_data tables. The query aggregates total purchase amounts and ranks customers based on their spending.

- The top 5 high-value customers are identified, along with their personal details (e.g., name, email, gender, date of birth) and total amount spent.
- **Customer 94** is the highest spender with **$7,880.24**, followed closely by **Customer 100** with **$7,808.09**.

```
Select c.first_name,c.last_name, c.email,c.gender,c.date_of_birth ,Round(sum(s.total_amount),2) as Amount_Spent
from customers_data c join sales_data s
on c.customer_id = s.customer_id
group by c.customer_id
order by Amount_spent desc
limit 5;
```

first_name	last_name	email	gender	date_of_birth	Amount_Spent
FirstName94	LastName94	customer94@example.cc	Female	1/31/1974	7880.24
FirstName100	LastName100	customer100@example.c	Female	11/6/1980	7808.09
FirstName92	LastName92	customer92@example.cc	Male	3/28/1987	7486.66
FirstName7	LastName7	customer7@example.cor	Male	1/8/1969	6712.69
FirstName16	LastName16	customer16@example.cc	Male	7/1/1999	6571.85

Details of Customers Born in the 1990s

This analysis focuses on customers born in the 1990s and provides insights into their total spending, total discounts received, and specific order details.

- **Total Spending**: The query computes the total spending (total_amount_spent) for each customer born in the 1990s.
- **Total Discounts**: Tracks the total discount applied across all orders for each customer.
- **Order-Level Details**: Provides details like quantity_sold, sale_date, and order_total for each transaction.

- **Customer 2** has made several purchases totaling **$6,059.56**, with cumulative discounts of **96%** across orders.

```
SELECT
    c.customer_id, c.first_name, c.last_name, c.email, c.gender, c.date_of_birth, c.registration_date,
    Round(SUM(s.total_amount) OVER (PARTITION BY c.customer_id), 2) AS total_amount_spent,
    Round(SUM(s.discount_applied) OVER (PARTITION BY c.customer_id), 2) AS total_discount_applied,
    s.quantity_sold,s.sale_date,s.discount_applied,s.total_amount AS order_total
FROM customers_data c JOIN sales_data s
ON c.customer_id = s.customer_id
WHERE YEAR(c.date_of_birth) BETWEEN 1990 AND 1999
ORDER BY c.customer_id, s.sale_date
LIMIT 1000;
```

customer_id	first_name	last_name	email	gender	date_of_birth	registration_date	total_amount_spent	total_discount_applied	quantity_sold	sale_date	discount_applied	order_total
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	2	1/2/2024	0	465.44
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	5	2/20/2024	20	1131.56
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	5	2/23/2024	15	150.2
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	5	3/12/2024	0	1645
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	4	3/22/2024	5	1074.98
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	2	3/27/2024	20	241.41
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	2	5/5/2024	0	372.04
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	3	6/28/2024	0	376.95
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	1	9/23/2024	15	158.12
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	4	10/4/2024	10	58.57
2	FirstName2	LastName2	customer2	Female	8/4/1991	11/19/2023	6059.56	95	1	11/3/2024	10	385.29
6	FirstName6	LastName6	customer6	Male	9/4/1992	1/4/2024	3467.22	40	1	1/1/2024	5	406.69
6	FirstName6	LastName6	customer6	Male	9/4/1992	1/4/2024	3467.22	40	1	1/7/2024	0	68.46
6	FirstName6	LastName6	customer6	Male	9/4/1992	1/4/2024	3467.22	40	2	4/3/2024	20	527.81
6	FirstName6	LastName6	customer6	Male	9/4/1992	1/4/2024	3467.22	40	4	4/12/2024	5	845.04
6	FirstName6	LastName6	customer6	Male	9/4/1992	1/4/2024	3467.22	40	1	5/3/2024	0	228.61
6	FirstName6	LastName6	customer6	Male	9/4/1992	1/4/2024	3467.22	40	4	9/4/2024	10	1390.61
10	FirstName10	LastName10	customer1	Male	5/21/1997	12/3/2023	4519.44	55	4	11/24/2023	20	442.56
10	FirstName10	LastName10	customer1	Male	5/21/1997	12/3/2023	4519.44	55	4	1/4/2024	20	1004.54
10	FirstName10	LastName10	customer1	Male	5/21/1997	12/3/2023	4519.44	55	2	2/20/2024	10	509.2
10	FirstName10	LastName10	customer1	Male	5/21/1997	12/3/2023	4519.44	55	5	5/3/2024	5	2100.74
10	FirstName10	LastName10	customer1	Male	5/21/1997	12/3/2023	4519.44	55	4	5/17/2024	0	357.52

Customer Segmentation:

This analysis categorizes customers into segments based on their total spending, such as **Low Spenders**, **Medium Spenders**, and **High Spenders**.

- **Low Spenders**: Customers with total spending less than $2,000.
- **Medium Spenders**: Customers with spending between $2,000 and $4,000.
- **High Spenders**: Customers with spending above $4,000.

```sql
SELECT
    c.customer_id, c.first_name, c.last_name, c.email, c.gender, c.date_of_birth,
    Round(SUM(s.total_amount),2) AS total_spent,
    CASE
        WHEN SUM(s.total_amount) < 2000 THEN 'Low Spender'
        WHEN SUM(s.total_amount) BETWEEN 2000 AND 4000 THEN 'Medium Spender'
        ELSE 'High Spender'
    END AS spending_segment
FROM Customers_data c JOIN Sales_data s ON c.customer_id = s.customer_id
GROUP BY
    c.customer_id, c.first_name, c.last_name, c.email, c.gender, c.date_of_birth
ORDER BY
    total_spent DESC;
```

customer_id	first_name	last_name	email	gender	date_of_birth	total_spent	spending_segment
94	FirstName94	LastName94	customer	Female	1/31/1974	7880.24	High Spender
100	FirstName100	LastName100	customer	Female	11/6/1980	7808.09	High Spender
92	FirstName92	LastName92	customer	Male	3/28/1987	7486.66	High Spender
7	FirstName7	LastName7	customer	Male	1/8/1969	6712.69	High Spender
16	FirstName16	LastName16	customer	Male	7/1/1999	6571.85	High Spender
60	FirstName60	LastName60	customer	Female	6/14/1994	6300.79	High Spender
17	FirstName17	LastName17	customer	Female	2/12/1990	6177.61	High Spender
95	FirstName95	LastName95	customer	Female	10/2/1982	6169.84	High Spender
2	FirstName2	LastName2	customer	Female	8/4/1991	6059.56	High Spender
28	FirstName28	LastName28	customer	Male	7/1/1971	5992.74	High Spender
68	FirstName68	LastName68	customer	Male	1/14/2006	5935.3	High Spender
35	FirstName35	LastName35	customer	Female	6/26/1985	5656.12	High Spender
33	FirstName33	LastName33	customer	Female	9/29/1987	5406.79	High Spender
5	FirstName5	LastName5	customer	Male	8/22/1979	5336.57	High Spender
31	FirstName31	LastName31	customer	Female	6/15/1988	5188.22	High Spender
47	FirstName47	LastName47	customer	Male	1/28/1967	4909.69	High Spender
18	FirstName18	LastName18	customer	Male	2/13/1977	4877.24	High Spender
23	FirstName23	LastName23	customer	Female	6/17/1962	4800.01	High Spender
27	FirstName27	LastName27	customer	Male	6/3/1995	4777.3	High Spender

High spenders represent the most valuable customer group for loyalty programs.

Module 3: Inventory and Product Management

Stock Management:

This analysis identifies products that are running low in stock (below a threshold of 10 units) and recommends the number of items to restock based on their past sales performance.

- **Low Stock Products**: Products with stock_quantity < 10 are flagged as running low.
- **Restocking Recommendation**:
  - The query calculates the **average daily sales** (avg_sales_per_day) for each product using past sales data.
  - For products running low, the recommended restocking amount is calculated as:

avg_sales_per_day * 30

```sql
WITH Recommend AS (
    SELECT
p.product_id, p.product_name, p.category, p.price, p.stock_quantity, a.avg_sales_per_day,
        CASE
            WHEN p.stock_quantity < 10 THEN a.avg_sales_per_day * 30
            ELSE 0
        END AS restock_items
    FROM Products_data p
    LEFT JOIN ( SELECT
            product_id, AVG(quantity_sold) AS avg_sales_per_day
        FROM Sales_data
        GROUP BY product_id
    ) a ON p.product_id = a.product_id
)

SELECT
    r.product_id, r.product_name, r.category, r.price, r.stock_quantity,r.avg_sales_per_day, r.restock_items
FROM Recommend r
WHERE r.stock_quantity < 10;
```

product_id	product_name	category	price	stock_quantity	avg_sales_per_day	restock_items
12	Product12	Sports	295.27	8	2.4286	72.858
16	Product16	Home	442.26	7	3.0909	92.727
41	Product41	Clothing	355.61	9	2.7	81

Inventory Movements Overview:

This report analyzes daily inventory movements for each product over the year 2024. The analysis captures the number of items restocked (IN) and sold (OUT) on each date.

- **total_restocked**: The sum of quantities added to inventory (IN) on a given date.
- **total_sold**: The sum of quantities removed from inventory (OUT) on the same date.

- For **Product 1**, inventory was restocked on **6/25/2024** with 19 units and 0 sold on that date.
- For **Product 2**, 17 units were restocked on **4/14/2024**, and 11 units were sold on **10/8/2024**.

```sql
-- Report for year 2024
SELECT
    i.product_id, p.product_name,p.category, i.movement_date,
    SUM(CASE WHEN i.movement_type = 'IN' THEN i.quantity_moved ELSE 0 END) AS total_restocked,
    SUM(CASE WHEN i.movement_type = 'OUT' THEN i.quantity_moved ELSE 0 END) AS total_sold
FROM Inventory_Movements_Data i
JOIN Products_data p ON i.product_id = p.product_id
WHERE i.movement_date BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY i.product_id, p.product_name, i.movement_date
ORDER BY i.product_id, i.movement_date;
```

product_id	product_name	category	movement_date	total_restocked	total_sold
1	Product1	Home	1/2/2024	1	0
1	Product1	Home	4/22/2024	0	4
1	Product1	Home	6/25/2024	19	0
1	Product1	Home	9/26/2024	7	0
2	Product2	Clothing	2/5/2024	11	0
2	Product2	Clothing	4/14/2024	17	0
2	Product2	Clothing	10/8/2024	0	11
2	Product2	Clothing	10/19/2024	0	15
3	Product3	Sports	8/8/2024	0	2
5	Product5	Sports	3/22/2024	0	7
5	Product5	Sports	9/27/2024	19	0
6	Product6	Sports	6/9/2024	0	17
6	Product6	Sports	7/30/2024	0	11
6	Product6	Sports	8/25/2024	5	0
6	Product6	Sports	9/26/2024	0	12
7	Product7	Sports	6/16/2024	2	0
8	Product8	Electronics	2/4/2024	0	5
8	Product8	Electronics	2/10/2024	2	0
8	Product8	Electronics	3/31/2024	0	14

Ranking Products by Price

This query categorizes products within each category based on their prices, assigning ranks such as **High**, **Middle**, and **Low**.

- **High Price**: Products with prices greater than or equal to **75% of the maximum price** within their category.
- **Middle Price**: Products with prices between **50% and 75% of the maximum price** in their category.

- **Low Price**: Products priced below **50% of the maximum price** in their category.

```sql
SELECT
    p.product_id, p.product_name, p.category, p.price,
    CASE
        WHEN p.price >= (SELECT MAX(price) FROM Products_data WHERE category = p.category) * 0.75 THEN 'High'
        WHEN p.price >= (SELECT MAX(price) FROM Products_data WHERE category = p.category) * 0.50 THEN 'Middle'
        ELSE 'Low'
    END AS price_rank
FROM Products_data p
ORDER BY p.category, p.price DESC;
```

product_id	product_name	category	price	price_rank
41	Product41	Clothing	355.61	High
26	Product26	Clothing	329.88	High
18	Product18	Clothing	313.92	High
17	Product17	Clothing	294.81	High
14	Product14	Clothing	280.73	High
25	Product25	Clothing	235.95	Middle
35	Product35	Clothing	232.72	Middle
2	Product2	Clothing	143.95	Low
34	Product34	Clothing	123.8	Low
42	Product42	Clothing	68.46	Low
49	Product49	Clothing	52.44	Low
13	Product13	Electronics	487.12	High
8	Product8	Electronics	450.01	High
37	Product37	Electronics	418.53	High
44	Product44	Electronics	411.58	High
15	Product15	Electronics	386.28	High
43	Product43	Electronics	366.49	High
40	Product40	Electronics	323.79	Middle
20	Product20	Electronics	245.61	Middle

Module 4: Advanced Analytics

Advanced Analytics – Average Order Size

The average order size analysis calculates the typical quantity of each product sold per transaction, providing insights into customer purchasing patterns.

```sql
SELECT
    s.product_id, p.product_name,
    AVG(s.quantity_sold) AS average_OrderSize
FROM Sales_data s
JOIN Products_data p ON s.product_id = p.product_id
GROUP BY s.product_id, p.product_name
ORDER BY p.product_name;
```

product_id	product_name	average_OrderSize
1	Product1	2.8571
10	Product10	2.9286
11	Product11	2.2143
12	Product12	2.4286
13	Product13	3.6
14	Product14	2.4615
15	Product15	3
16	Product16	3.0909
17	Product17	3.6
18	Product18	3.0588
19	Product19	2.8889
2	Product2	2.1429
20	Product20	3.0556
21	Product21	3
22	Product22	4.2
23	Product23	2.6667
24	Product24	3
25	Product25	2.8182
26	Product26	2.7692

Advanced Analytics – Recent Restock Products

This analysis identifies the products that have been restocked most recently, providing insights into inventory replenishment trends. The query retrieves the latest restock date (movement_type = 'IN') for each product by examining the Inventory_Movements_data table and highlights those that have undergone recent stock adjustments.

```
SELECT
    m.product_id, p.product_name,p.category,p.price,
    MAX(m.movement_date) AS Latest_date_of_Restocked
FROM Inventory_Movements_data m
JOIN Products_data p ON m.product_id = p.product_id
WHERE m.movement_type = 'IN'
GROUP BY m.product_id, p.product_name
ORDER BY Latest_date_of_Restocked DESC;
```

product_id	product_name	category	price	Latest_date_of_Restocked
24	Product24	Sports	351.65	11/1/2024
22	Product22	Sports	174.01	10/9/2024
40	Product40	Electronics	323.79	10/6/2024
5	Product5	Sports	249.53	9/27/2024
1	Product1	Home	429.6	9/26/2024
13	Product13	Electronics	487.12	9/17/2024
46	Product46	Home	222.38	9/14/2024
31	Product31	Sports	76.27	9/13/2024
20	Product20	Electronics	245.61	9/3/2024
44	Product44	Electronics	411.58	8/29/2024
6	Product6	Sports	139.55	8/25/2024
49	Product49	Clothing	52.44	8/24/2024
26	Product26	Clothing	329.88	8/21/2024
19	Product19	Home	150.88	8/14/2024
15	Product15	Electronics	386.28	8/12/2024
9	Product9	Sports	249.23	8/7/2024
41	Product41	Clothing	355.61	8/5/2024
35	Product35	Clothing	232.72	8/3/2024
45	Product45	Electronics	144.7	7/10/2024

Seasonal Trends

This analysis examines seasonal trends by analyzing monthly purchase patterns. The query calculates the total number of purchases (total_purchases) and the average purchase value (avg_purchase_value) for each month by examining the Inventory_Movements_data table.

- Months with higher total purchases and average purchase values indicate peak sales periods.

```sql
-- Seasonal trends
SELECT
    MONTH(s.sale_date) AS purchase_month,
    COUNT(s.sale_id) AS total_purchases,
    ROUND(AVG(s.total_amount), 2) AS avg_purchase_value
FROM sales_data s
GROUP BY MONTH(s.sale_date)
ORDER BY purchase_month;
```

ult Grid | Filter Rows: | Export: | Wrap Cell Content:

purchase_month	total_purchases	avg_purchase_value
1	38	588.1
2	27	580.8
3	41	609.13
4	38	650.11
5	53	616.82
6	44	619.06
7	45	718.15
8	41	744.99
9	42	516.22

Customer Purchase Patterns

This analysis tracks customer purchase behavior by calculating the time gap between consecutive purchases. The query retrieves the **previous purchase date** for each customer and calculates the **days since the last purchase** using the DATEDIFF () function using the LAG () window function.

```sql
SELECT
    customer_id,
    sale_date,
    LAG(sale_date) OVER (PARTITION BY customer_id ORDER BY sale_date) AS previous_PurchaseDate,
    DATEDIFF(sale_date, LAG(sale_date) OVER (PARTITION BY customer_id ORDER BY sale_date)) AS days_since_last_purchase
FROM Sales_data
ORDER BY customer_id, sale_date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	sale_date	previous_purchase_date	days_since_last_purchase
1	2023-11-09	NULL	NULL
1	2024-02-15	2023-11-09	98
1	2024-03-12	2024-02-15	26
1	2024-03-13	2024-03-12	1
1	2024-06-03	2024-03-13	82
2	2024-01-02	NULL	NULL
2	2024-02-20	2024-01-02	49
2	2024-02-23	2024-02-20	3
2	2024-03-12	2024-02-23	18